

Verifier-Guided Evaluation of LLM-Generated Network Configurations: a Paired Study with Shared Initial Candidates

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory study (CAND-2026-001) that measured whether a verifier-triggered guarded pipeline (generate $\rightarrow$ validate $\rightarrow$ repair, ≤ 1 repair) reduces deterministic-invariant violations relative to single-shot initial generation for intent $\rightarrow$ configuration NetOps tasks. Using a deterministic synthetic task generator and a deterministic network verifier, we ran 200 preregistered tasks under shared_initial_candidate semantics with the hosted model gpt-5-mini (execution/model_configuration.json records temperature = null/unset). Baseline configurations passed the verifier in 199/200 tasks (99.5%); the guarded pipeline passed 200/200 (100%). Paired absolute risk difference (guarded - baseline) = 0.005; contingency counts $n_{11}=199, n_{10}=1, n_{01}=0, n_{00}=0$; exact two-sided McNemar $p = 1.0$; paired-bootstrap 95% percentile CI = [0.00, 0.015] (10,000 resamples, seed 1729). The single guarded improvement arose from one verifier-triggered repair that corrected a multi-step ordering violation. We report preregistration, full execution accounting (201 model calls: 200 shared initial + 1 repair), paired analysis, representative diagnostic artifacts, and bounded limitations grounded in the archived execution bundle.

I. INTRODUCTION

Generative large language models (LLMs) are increasingly proposed to translate high-level operator intent into device configurations for network operations (NetOps). Operational correctness for such workflows requires generated configurations to satisfy deterministic invariants (reachability, policy compliance, workflow ordering, and group-level safety). Prior prototype systems and benchmarks illustrate feasibility but leave open questions about how much deterministic verification and bounded repair alter acceptance rates when the identical initial sample is reused across comparison conditions.

This paper reports a preregistered paired experiment (CAND-2026-001) designed to isolate the marginal effect of a verifier-triggered automated repair on an identical generated candidate. Under shared_initial_candidate semantics the same single initial generation is deterministically reused by both baseline and guarded episodes; any paired difference therefore arises solely from the permitted validator-triggered repair. The primary estimand is the paired difference in per-task binary acceptance by a deterministic verifier (paired_success_rate_difference_guarded_minus_baseline). Because shared_initial_candidate semantics were enforced, this estimand measures a repair-only effect and does not capture potential benefits from independent guarded first-pass generations, constrained prompts, or multi-sample selection.

Contributions. (1) A preregistered, fully archived paired experiment executing 200 intent $\rightarrow$ configuration tasks with a deterministic verifier and a hosted LLM (gpt-5-mini) under shared_initial_candidate semantics. (2) Complete execution accounting and deterministic reconciliation showing 200 shared initial generations and 1 repair call (201 model calls total). (3) Artifact-grounded confirmatory analysis reporting baseline pass 199/200 and guarded pass 200/200 with contingency counts ($n_{11}=199, n_{10}=1, n_{01}=0, n_{00}=0$), paired risk difference 0.005, exact two-sided McNemar $p = 1.0$, and bootstrap 95% CI = [0.00, 0.015] (10,000 resamples, seed 1729). (4) A localized failure diagnosis for the single repaired pair using archived validator traces. (5) Detailed reproducibility guidance and bounded limitations tied to archived artifacts and reconciliation records.

II. RELATED WORK

Recent work on LLM-driven NetOps shows convergent interest in intent $\rightarrow$ configuration synthesis, verification-aware pipelines, and tool-augmented agent designs, but the supplied verified records reveal complementary strengths and limitations that motivated our paired design. NetConfEval and related intent $\rightarrow$ configuration prototypes demonstrate that LLMs can produce workable device configurations from natural-language intent and that task-style benchmarks are useful for early evaluation; these studies report feasibility and dataset-limited performance measures but do not uniformly integrate a deterministic verifier as the final oracle for acceptance decisions [1]. Deterministic configuration verification methods (e.g., formal and symbolic reachability/policy-check approaches) provide a clear operational oracle for syntactic and semantic invariants and have been developed in the NetOps literature; these verification techniques form the natural oracle used in our experiment and highlight why a verifier-centric evaluation is defensible for configuration-synthesis tasks [2].

Across adjacent domains, generate $\rightarrow$ validate $\rightarrow$ repair patterns have empirical support: test-in-the-loop program-repair and synthesis-with-tests pipelines show that validating generated artifacts against executable checks and iteratively repairing failures materially improves correctness in software-engineering tasks [4]. Those demonstrations motivate guarded NetOps pipelines but are not direct evidence of net effects in network-configuration contexts because NetOps invariants and multi-device sequencing introduce distinct dependency

patterns. Tool-augmented agent and model-context approaches further extend capability by granting controlled access to specialized solvers, validators, or execution environments; evaluations of such tool-augmented designs argue they enable more complex, verifiable behavior but also point out engineering and interoperability challenges when integrating heterogeneous oracles [6].

Survey and reporting-guideline literature places additional context around rigor and reproducibility: broad LLM surveys synthesize model capabilities and limitations at scale and underscore the importance of careful experimental reporting and preregistration when evaluating generative systems [5]. Reporting guidelines for studies that use LLMs emphasize explicit documentation of sampling, validation, and analysis choices to avoid overclaiming—this influenced our preregistration and artifact-archiving decisions [3]. Finally, focused work on hallucination detection and validator gating in generative systems highlights common failure modes (confabulation, ordering errors, and adversarially induced fabrications) and motivates verifier gates as necessary but not sufficient mitigations; such analyses support our guarded pipeline as a targeted, bounded intervention rather than a general cure [7].

Synthesis. The supplied and verified literature collectively supports three points: LLM-based intent→config pipelines are feasible and actively prototyped; generate→validate→repair is a promising architectural pattern whose empirical demonstrations to date are stronger in code and testable artifacts than in NetOps-specific verifier studies; and rigorous reporting and verifier-grounded evaluations are necessary to interpret operational claims. Our paper addresses a narrower, previously underexplored empirical gap in the supplied records: a reproducible paired measurement of the marginal (repair-only) effect when the identical initial candidate is reused across baseline and guarded conditions. By enforcing shared_initial_candidate semantics and archiving the verifier traces, the present study isolates that repair-only estimand and complements prior prototype and generate–validate–repair demonstrations [1],[2],[4],[6],[3],[5].

III. METHODOLOGY

Preregistration and design freeze. We preregistered study CAND-2026-001 and froze the confirmatory design prior to model calls. The preregistration (preregistration_sha256 recorded in the execution manifest) fixed the primary estimand, the paired design, the selection rule for the deterministic task sample, generation semantics, maximum repair calls, the primary confirmatory test, and the missingness/treatment rules. The study_id is CAND-2026-001 and the execution contract declared generation_semantics = shared_initial_candidate, initial_generation_calls_per_task = 1, and maximum_repair_calls_per_task = 1.

Deterministic task selection and manifest encoding. The 200 tasks were selected deterministically from a synthetic task bank produced by deterministic_netops_task_generator_v5 according to the preregistered index rule (for zero-based ordinal j in 0..199: cycle = $j // 5$; offset = [4,5,6,7,8][j

% 5]; task_index = $8 * \text{cycle} + \text{offset}$). Each task manifest (execution/task_manifest.jsonl) encodes: initial_state, expected_final_state, required_sequence (an ordered command list that the verifier enforces), allowed_touched_fields, workflow_policies (e.g., group constraints and minimum-active requirements), difficulty metadata (changed_interface_count, dependency_rule_count, required_command_count), and a normalized reference_answer. Representative tasks in the archive show required_command_count $\in \{3, \dots, 9\}$ and difficulty labels very_hard or extreme; these manifest fields are authoritative inputs to the deterministic_netops_validator_v5 scoring logic.

Model configuration and sampling semantics. The requested hosted model is declared as gpt-5-mini in execution/model_configuration.json. That artifact records provider = openai_responses, declared_model_version = gpt-5-mini, max_output_tokens = 1500, maximum_attempts_per_call = 1, reasoning_effort = "minimal", store = false, and temperature = null (unset). We explicitly note that temperature = null/unset does not imply deterministic sampling or temperature = 0; nondeterministic hosted-model sampling may still occur. The preregistered adapter-level contract enforces shared_initial_candidate semantics: a single initial generation call per task was deterministically reused by both baseline and guarded episodes (no independent per-condition sampling). The run’s deterministic reconciliation (archived) reports stage_counts = {shared_initial_generation: 200, repair: 1} and hosted_model_call_event_count = 201, confirming the planned call pattern.

Sanitization, exclusions, and missingness handling. The preregistration specified an automatic syntax sanitizer (automatic_syntax_sanitizer_v1) to deterministically correct minor formatting issues prior to verifier parsing. Exclusion rules allowed deterministic pre-call exclusion only when a vendor-template token-normalized match indicated potential leakage or if the provided input failed syntactic pre-checks; such deterministic exclusions would be logged. The missingness_plan declared primary failed-call handling (complete_pair_primary) and a sensitivity treatment that would treat persistent unparseables as failures. In the archived run no preregistered exclusions occurred and there were no persistent unparseables; missingness artifacts (analysis/missingness_summary.csv) record complete_pairs = 200 with zeros elsewhere.

Deterministic verifier, scoring rule, and validator traces. The oracle was deterministic_netops_validator_v5 (validator_version 5.0); it implements a deterministic parser and an invariant checker that enforces per-task constraints encoded in each task manifest. The verifier’s full_intent_constraint_validation returns 1 only when: (a) all required_sequence ordering constraints are satisfied, (b) allowed_touched_fields are respected, (c) group-level invariants (final_group_constraints, minimum_active_in_groups) hold, and (d) no protected interfaces or preserved fields are modified. Per-episode validator_trace artifacts (archived under execution/scoring/) include validation_before

and validation_after objects, final_configuration, normalized_configuration, parsed_command_count, required_command_count, observed_touched_field_count, violation_count, violation_codes, validator_id, and validation boolean valid. These traces are the authoritative evidence for per-episode scores and repair decisions.

Paired execution semantics and repair mechanism. Under shared_initial_candidate semantics the identical sampled candidate (shared_initial_candidate) is provided to the baseline and guarded episodes. The guarded pipeline permits at most one automatic repair call triggered when the verifier reports violations; the repair is implemented as a single repair prompt to the hosted model (repair stage_count = 1 observed). Because baseline and guarded reuse the same initial candidate, any paired success difference can only arise from the permitted validator-triggered repair. This design isolates the repair-only estimand and does not measure potential benefits from independent guarded-first-pass generations or constrained prompting variants.

Statistical analysis plan and exact-test implementation. The preregistered primary test for the binary paired outcome (all-specified-invariants-pass) is the exact two-sided McNemar test computed from discordant pair counts n_{10} (guarded=1, baseline=0) and n_{01} (guarded=0, baseline=1). We implement the exact two-sided McNemar via the exact binomial tail: for $n = n_{10} + n_{01}$ and $k = n_{10}$, compute $p = 2 * \min\{\text{BinomCDF}(k; n, 0.5), 1 - \text{BinomCDF}(k - 1; n, 0.5)\}$ with conventional handling for $k = 0$. Paired-bootstrap percentile confidence intervals for the absolute risk difference (guarded - baseline) were planned as an auxiliary estimator; archived analysis used bootstrap_resamples = 10000 and bootstrap_seed = 1729 (parameters recorded in analysis/analysis_log.jsonl). The primary inference uses complete_pair_primary handling; the preregistered sensitivity treating persistent unparsables as failures is supported by the archived artifacts but was not needed for this run.

Reproducibility and artifact linkage. The preregistration, model_configuration.json, task manifests, raw model responses (execution/responses/), per-episode scoring artifacts (execution/scoring/), the deterministic reconciliation (analysis/deterministic_reconciliation.json), and the analysis artifacts (analysis/*.csv, analysis/analysis_log.jsonl) together provide the machine-verifiable evidence trace for every methodological choice described above. The initial master prompt is archived (provenance/master_prompt.txt; SHA-256 recorded) and the execution manifest lists representative artifact hashes; these archived items are the authoritative sources for the methodological parameters reported here.

IV. RESULTS

Execution accounting and provider audit. The run started at 2026-09-01T16:03:05.491039+00:00 and completed at 2026-09-01T16:32:13.298536+00:00 (≈ 29.2 minutes wall-clock). The execution manifest records planned_episode_count = 400 and completed_episode_count = 400 with failed_episode_count = 0. Provider-call

accounting (deterministic_reconciliation.json) reports hosted_model_call_event_count = 201, with stage_counts = {shared_initial_generation: 200, repair: 1} and condition_counts reported as {shared: 200, guarded: 1} under the adapter’s stage/condition accounting semantics. Cache statistics show cache_miss_count = 201 and cache_hit_count = 0, consistent with fresh shared-initial generations for all tasks plus a single repair call. Dividing the wall-clock duration by the 201 hosted-model calls yields an approximate average model-call latency of 8.7 seconds per call (simple elapsed-time / call count calculation from archived timestamps and model_calls_used).

Primary confirmatory outcomes (archived CSV artifacts). The archived condition summary (analysis/condition_summary.csv) records baseline successes = 199, baseline failures = 1 (success_rate = 0.995) and guarded successes = 200, guarded failures = 0 (success_rate = 1.000). The paired contingency table (analysis/paired_contingency_table.csv) contains the exact cell counts used by the confirmatory test: $n_{11} = 199$ (both pass), $n_{10} = 1$ (guarded pass only), $n_{01} = 0$ (baseline pass only), $n_{00} = 0$ (both fail). From these counts the observed paired absolute risk difference (guarded - baseline) = 0.005.

Exact-test and interval quantification. Applying the exact two-sided McNemar (binomial-tail formulation) to the discordant counts ($n = n_{10} + n_{01} = 1$, $k = n_{10} = 1$) yields $p = 1.0$ as computed in Methods. To quantify plausible effect sizes we computed a paired-bootstrap percentile 95% confidence interval for the absolute risk difference using 10,000 resamples with bootstrap_seed = 1729 (analysis/analysis_log.jsonl). The resulting 95% percentile CI is [0.00, 0.015], giving a bounded estimate consistent with at most small absolute improvements under the repair-only estimand given the observed data.

Missingness, contamination, and deterministic reconciliation. The run recorded no preregistered exclusions and no persistent unparsables; analysis/missingness_summary.csv shows complete_pairs = 200 and zeros for all other missingness categories. analysis/contamination_summary.csv contains no flagged contamination entries. The deterministic reconciliation artifact (analysis/deterministic_reconciliation.json) reports pair_accounting_consistent = true and all_deterministic_consistency_checks_passed = true, confirming internal accounting consistency between provider-call logs, stage_counts, and analysis inputs.

Localized failure diagnosis and repair evidence. The guarded pipeline invoked exactly one automated repair (stage_counts shows repair = 1). Validator traces for the repaired pair (archived under execution/scoring/) indicate an extreme composite task whose manifest metadata records required_command_count = 9 and changed_interface_count = 3. The shared initial candidate for that pair violated the manifest’s required_sequence ordering constraint; validation_before.violation_codes and validation_before.normalized_configuration recorded the pre-repair mismatch. The repair interaction produced a

localized reordering of commands that matched the manifest’s `required_sequence` and yielded `validation_after.valid = true`. The evidence supporting this sequence of events is entirely artifact-grounded: the per-episode `validator_trace` fields (`validation_before.violation_codes`, `parsed_command_count`, `required_command_count`, `normalized_configuration`, `repair_applied` flag, and `validation_after.*`) and the repair provider trace are archived and indexable. These traces show that the repair fixed an ordering/dependency violation expressible in the task manifest’s invariants rather than a syntactic parsing error.

Exploratory diagnostics by difficulty metadata. Representative task manifests available in the execution bundle document `required_command_count` values ranging from 3 up to 9 and difficulty levels labeled `very_hard` or `extreme`. The single baseline failure (the repaired pair) is associated with the highest recorded complexity in the representative sample (`required_command_count = 9`, `changed_interface_count = 3`), consistent with the preregistered exploratory hypothesis that multi-device sequencing and dependency complexity concentrate residual failures. No other tasks in the archived representative subset exhibited violations requiring repair; many `very_hard` tasks (`required_command_count` in the mid-range) were validated successfully without repair.

Practical reproducibility pointers for confirming results. All artifacts needed to reproduce the primary numbers are archived and referenced by the execution manifest: `analysis/paired_contingency_table.csv` and `analysis/condition_summary.csv` contain the per-condition counts; `analysis/analysis_log.jsonl` contains the bootstrap parameters (`resamples=10000`, `seed=1729`); `deterministic_reconciliation.json` and `execution/model_configuration.json` record provider-call accounting and declared model fields (including `temperature = null`); `execution/responses/` and `execution/scoring/` contain the shared-initial responses and per-episode `validator_trace` objects demonstrating `validation_before` and `validation_after` states and the single repair interaction. Re-running the archived `paired_binary_analysis_v1` executor on the archived `input_results_sha256` reproduces the confirmatory outputs (`analysis/results.json`) without recomputation of model calls.

Summary of empirical evidence. The archived run shows that under `shared_initial_candidate` semantics (the same sampled candidate reused by both conditions) and with the chosen synthetic task bank and deterministic verifier, nearly all initial single-shot candidates already satisfied the encoded deterministic invariants (199/200). Allowing a single verifier-triggered bounded repair corrected one manifest-expressed ordering violation in an extreme composite task, producing the observed paired improvement ($n_{10} = 1$). The small number of discordants places the inferential regime in a sparse-discordant setting where the exact McNemar test yields $p = 1.0$ but the bootstrap CI bounds the plausible absolute improvement to at most ~ 1.5 percentage points under the paired repair-only estimand.

V. DISCUSSION

We expand the interpretive analysis and diagnostics here while preserving the paper’s existing supported content.

Estimand interpretation and scope (explicit). Because the experiment used `shared_initial_candidate` semantics, baseline and guarded episodes for each task began from the identical sampled candidate; consequently the primary estimand intentionally measures the marginal effect of permitting an automated verifier-triggered repair on that identical candidate (a repair-only effect). This design isolates repair mechanics from any potential benefits that could arise if the guarded condition were allowed to sample independently or draw multiple candidates.

Sparse-discordant regime and inferential consequences. The observed data ($n_{11}=199$, $n_{10}=1$, $n_{01}=0$) place the analysis in a sparse-discordant regime, a setting where conventional asymptotic approximations are unreliable and exact discrete tests or resampling are appropriate. Our use of the exact binomial-tail McNemar calculation and a paired-bootstrap percentile CI (10,000 resamples, seed 1729 as archived in `analysis/analysis_log.jsonl`) is consistent with best practice for small discordant counts. Practically, the realized baseline success rate (99.5%) substantially reduced the experiment’s sensitivity to detect modest repair-only effects relative to our preregistered power target (designed for mid-range baseline rates $\sim 45\text{--}65\%$). The bootstrap CI $[0.00, 0.015]$ quantifies that the data are compatible with effects up to 1.5 percentage points in absolute terms under our paired repair-only estimand; the preregistered design could not reliably detect larger effects because none were present.

Mechanistic diagnostic of the repaired episode. The single repair call corrected a concrete sequencing violation expressible in the task manifest’s `required_sequence` field (`required_command_count = 9` in the repaired manifest). Archived `validator_trace` fields show `validation_before.violation_codes` listing the ordering breach, `parsed_command_count` equal to the presented commands, and `validation_after.valid = true` after the automated reordering repair. These per-episode traces demonstrate: (1) the verifier’s capacity to detect ordering invariants encoded in the manifest; (2) the repair module’s ability to perform localized, semantics-preserving reorderings that the verifier accepts; and (3) that such faults concentrated in tasks with higher `required_command_count` and extreme difficulty labels in our deterministic task bank. All diagnostic fields cited above are available in `execution/scoring/task-00000X-*.json` in the archived bundle.

Construct validity of the deterministic verifier. `deterministic_netops_validator_v5` enforces a structured subset of operational invariants encoded in the task payloads (ordering, `allowed_touched_fields`, group-level up/down constraints, and explicit per-field prerequisites). This verifier therefore provides a high-fidelity oracle for manifest-encoded invariants but does not emulate dataplane behavior, vendor runtime idiosyncrasies, or external control-plane side-effects. Construct-valid

conclusions should therefore be framed strictly in terms of manifest-specified, deterministically verifiable invariants; the experiment does not claim to validate dataplane reachability or vendor CLI execution semantics that lie outside the verifier’s encoding.

Design implications for future NetOps evaluations. The paired `shared_initial_candidate` design cleanly isolates repair capability; however, operational questions remain that require alternative designs: (a) independent guarded-first-pass sampling would quantify whether guarded prompting or constrained generation yields a higher baseline of valid samples before repair; (b) multi-sample selection (n-best) could reveal selection-improvement effects; and (c) multi-model comparisons would quantify model-dependent failure patterns. From an experimental-planning perspective our results highlight that preregistered power calculations must account for plausible high baseline performance—if baseline rates turn out near 100% the discordant-pair approach will often be underpowered to detect small absolute repairs-only gains.

Reproducibility, artifact usage, and recommended reanalyses. All artifacts necessary to reproduce analysis are archived: `execution/task_manifest.jsonl` (deterministic task selection and `required_sequence` fields), `execution/model_configuration.json` (declared model, noting `temperature = null/unset`), `execution/responses/` (shared-initial and per-condition responses), `execution/scoring/` (per-episode validator traces), `analysis/paired_contingency_table.csv` and `analysis/condition_summary.csv`, `deterministic_reconciliation.json` (provider accounting and `stage_counts`), and `analysis/analysis_log.jsonl` (bootstrap parameters and seed). Re-running the `paired_binary_analysis_v1` executor on the archived `input_results_sha256` reproduces the confirmatory results. For targeted reanalysis, researchers can (i) recompute bootstrap intervals under alternative resampling schemes, (ii) treat the single repaired task as an outlier to study robustness, or (iii) reclassify manifest invariants to evaluate sensitivity of verifier coverage—all without altering original model-call artifacts.

Threats to validity revisited (artifact-grounded). Internal validity: `shared_initial_candidate` semantics intentionally eliminate differences due to separate sampling, but they also prevent measuring effects of guarded prompt variants; model sampling nondeterminism remains possible because `temperature` was `unset` (`execution/model_configuration.json` records `temperature = null`). Construct validity: the verifier enforces only encoded invariants; omissions in the manifest (for example, dataplane-dependent invariants) are not tested. External validity: a single hosted model (gpt-5-mini) and a synthetic deterministic task bank limit generalizability; results do not imply identical outcomes for other models, live devices, or telemetry-driven validation. Conclusion validity: with one discordant pair the statistical evidence for a nonzero repair-only effect is weak ($p = 1.0$), though the bootstrap CI constrains plausible magnitudes.

Operational implications and cautious hypotheses. Artifact-grounded evidence demonstrates that bounded verifier-

triggered repairs can fix localized ordering violations that are detectable and expressible in task manifests. From an operational standpoint this suggests a practical pattern: require explicit, machine-checkable manifest invariants for change tasks and pair LLM synthesis with deterministic validation/repair gates before human or automated deployment. We emphasize these are actionable hypotheses supported by the archived diagnostics in this controlled synthetic setting; they are not operational claims that bounded repair will resolve dataplane-level or adversarially induced failures without further evaluation.

Summary of expanded diagnostics. The archived execution bundle provides the evidentiary chain: deterministic task manifests encoding the invariants, the shared initial generated candidates, per-episode validator traces showing before/after states, provider-call accounting confirming `shared_initial_generation = 200` and `repair = 1`, and analysis artifacts (contingency table, bootstrap parameters) documenting the statistical regime. Together these artifacts ground the narrower, repair-only conclusion and illuminate concrete design decisions for future, broader evaluations (independent guarded sampling, multi-model studies, live emulation).

VI. LIMITATIONS

- Synthetic deterministic task bank and verifier: results reflect correctness under the chosen synthetic intent templates and deterministic verifier and do not guarantee similar performance on live heterogeneous production devices or telemetry.
- Single-model experiment (gpt-5-mini): findings do not demonstrate multi-model generality.
- `Shared_initial_candidate` semantics: baseline and guarded conditions began from the same initial generation; the study measures verifier/repair effects only and does not evaluate independent first-pass generation contrasts.
- Limited adversarial/contamination testing: the run did not include systematic adversarial retrieval or poisoned-context attacks; such threats remain an open evaluation axis.
- Oracle coverage: the deterministic verifier enforces encoded invariants but may omit runtime behaviors and device-specific semantics observable only through live execution; external validity to live deployments is therefore limited.

VII. CONCLUSION

We executed a preregistered paired experiment measuring the marginal effect of a verifier-triggered repair under `shared_initial_candidate` semantics for LLM-generated network configurations. In 200 synthetic deterministic tasks with gpt-5-mini and `deterministic_netops_validator_v5`, baseline acceptance was 199/200 and guarded acceptance was 200/200; a single automated repair corrected a multi-device ordering violation. Paired absolute risk difference = 0.005 with paired-bootstrap 95% CI [0.00, 0.015] (10,000 resamples,

seed 1729) and exact two-sided McNemar $p = 1.0$. Artifact-grounded evidence therefore supports a constrained conclusion: verifier-guarded bounded repair can correct localized ordering faults in this synthetic verifier-backed setting. Broader operational claims require additional experiments: multi-model studies, independent-first-pass guarded semantics, adversarial contamination testing, and live-device or dataplane emulation to capture runtime semantics not modeled by the deterministic verifier.

DISCLOSURE STATEMENT

Disclosure: This run implemented preregistered study CAND-2026-001 and was executed autonomously after the autonomy lock. Declared model: gpt-5-mini (execution/model_configuration.json records temperature = null/unset; null/unset is not evidence of deterministic sampling or temperature = 0). Orchestration adapter: hosted_netops_gvr_v1. Exact initial master prompt archived at provenance/master_prompt.txt (SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582b b39b15ba). Preregistration fixed generation_semantics = shared_initial_candidate (one sampled initial generation per task was deterministically reused by baseline and guarded episodes) and allowed at most one automated repair call per task. Model-call accounting in the archived execution manifest reflects 200 shared initial generation calls and 1 repair call (201 model calls total). The deterministic verifier used was deterministic_netops_validator_v5. Humans selected the broad topic family and constrained the initial master prompt prior to the autonomy lock as permitted by the track; no human manually edited technical manuscript content after the autonomy lock. All provider traces, verifier logs, task manifests, model-configuration records, and analysis artifacts are archived in the execution bundle and indexed by the execution manifest.

REFERENCES

- [1] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296
- [2] Ryan Beckett, Aarti Gupta, Ratul Mahajan, David Walker, "A General Approach to Network Configuration Verification", 2017, 10.1145/3098822.3098834
- [3] Jack Gallifant, Majid Afshar, Saleem Ameen, Yindalon Aphinyanaphongs, Shan Chen, Giovanni Cacciamani, Dina Demner-Fushman, Dmitriy Dligach, Roxana Daneshjou, Chrystinne Oliveira Fernandes, Lasse Hyldig Hansen, Adam Landman, Lisa Soleymani Lehmann, Liam G. McCoy, Timothy A. Miller, Amy C. Moreno, Nikolaj Munch, David Restrepo, Guergana Savova, Renato Umeton, Judy Wawira Gichoya, Professor Gary S. Collins, Karel G.M. Moons, Leo Anthony Celi, Danielle S. Bitterman, "The TRIPOD-LLM reporting guideline for studies using large language models", 2025, 10.1038/s41591-024-03425-5
- [4] Yunhe Li, "Test-in-the-loop LLM Repair: Verifiable Automated Program Repair on QuixBugs with a "Failing Test $\rightarrow$ Patch $\rightarrow$ Regression Test" Loop", 2024, 10.69987/jacs.2024.40206
- [5] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Zican Dong, Yupeng Hou, Beichen Zhang, Yingqian Min, Junjie Zhang, Peiyu Liu, Xiaolei Wang, Yifan Du, Yushuo Chen, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Peiyu Liu, Yiwen Hu, Jian-Yun Nie, Ji-Rong Wen, "A Survey of Large Language Models", 2026, 10.1007/s11704-026-60308-3

- [6] Haoran Dong, "Tool-Augmented Large Language Model Agents for Analysis: A Focused Study", 2026, 10.2139/ssrn.6647800